Laporan Dataset Adult (Census Income): Klasifikasi Pendapatan dengan Naive Bayes

(Disusun Untuk Memenuhi Mata Kuliah Pembelajaran Mesin)

Dosen Pengampu:
Nelly Indriani Widiastuti, M.T.

Disusun Oleh:
Hilyati Raihatul Jannah - 10124053

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK DAN ILMU KOMPUTER
UNIVERSITAS KOMPUTER INDONESIA

1. Deskripsi Data

Sumber Data:

Dataset	https://archive.ics.uci.edu/dataset/2/adult
Ekstraksi Dataset	https://www.kaggle.com/datasets/uciml/adult-census-income
Makalah Ilmiah	https://cdn.aaai.org/KDD/1996/KDD96-033.pdf
Link Program	https://github.com/hilyatiliaa/Klasifikasi-Pendapatan

Dataset Adult (Census Income) merupakan dataset yang digunakan untuk menganalisis dan mengklasifikasikan tingkat pendapatan individu berdasarkan karakteristik demografis, pendidikan, dan pekerjaan. Dataset ini berasal dari ekstraksi data sensus Amerika Serikat tahun 1994 yang dilakukan oleh Barry Becker. Dataset tersebut diperkenalkan melalui UCI Machine Learning Repository dan banyak digunakan dalam penelitian serta pengembangan model klasifikasi data.

Dataset ini memiliki 32.561 baris data dan 15 kolom pada berkas yang digunakan dalam penelitian ini. Terdiri dari 14 fitur prediktor dan satu variabel target, yaitu income. Target tersebut memiliki dua kategori, yaitu <=50K dan >50K, yang menunjukkan kelompok pendapatan tahunan tidak lebih dari US$50.000 dan di atas US$50.000. Variabel yang tersedia meliputi usia (age), jenis pekerjaan (workclass), pendidikan (education), status perkawinan (marital.status), pekerjaan (occupation), jenis kelamin (sex), jumlah jam kerja per minggu (hours.per.week), serta beberapa atribut lainnya.

2. Fitur pada Dataset

No	Fitur	Deskripsi
1	age	Usia individu dalam tahun
2	workclass	Kategori tempat atau jenis sektor pekerjaan
3	fnlwgt	Bobot statistik yang berkaitan dengan estimasi jumlah populasi yang diwakili suatu observasi
4	education	Tingkat pendidikan yang telah dicapai
5	education.num	Representasi numerik tingkat pendidikan
6	marital.status	Status perkawinan individu
7	occupation	Jenis pekerjaan atau bidang pekerjaan individu
8	relationship	Kategori hubungan individu dalam rumah tangga
9	race	Kategori ras sebagaimana dicatat dalam data sensus

10	sex	Jenis kelamin yang tercatat dalam dataset
11	capital.gain	Nilai keuntungan modal
12	capital.loss	Nilai kerugian modal
13	hours.per.week	Jumlah jam kerja per minggu
14	native.country	Negara asal individu
15	income	Target klasifikasi pendapatan

Fitur dalam dataset Adult Income mencakup karakteristik demografis, pendidikan, pekerjaan, dan kondisi ekonomi individu. Sebanyak 14 fitur prediktor digunakan untuk mengenali pola pendapatan, sedangkan income menjadi fitur target yang mengelompokkan individu ke dalam kategori <=50K dan >50K

3. Jenis Data Berdasarkan Skala Pengukuran

Fitur	Tipe Data	Skala Pengukuran
age	Numerik	Rasio
workclass	Kategorikal	Nominal
fnlwgt	Numerik	Rasio
education	Kategorikal	Ordinal
education.num	Numerik diskret	Ordinal
marital.status	Kategorikal	Nominal
occupation	Kategorikal	Nominal
relationship	Kategorikal	Nominal
race	Kategorikal	Nominal
sex	Kategorikal	Nominal
capital .gain	Numerik	Rasio
capital.loss	Numerik	Rasio
hours.per.week	Numerik	Rasio
native.country	Kategorikal	Nominal
income	Kategorikal	Ordinal (biner)

Skala pengukuran yang digunakan adalah nominal, ordinal, dan rasio, tanpa skala interval. Skala nominal digunakan untuk kategori tanpa urutan, sedangkan skala ordinal digunakan pada education dan education.num karena menunjukkan jenjang pendidikan. Meskipun berbentuk angka, education.num hanya merupakan kode urutan dan jarak antarjenjang belum tentu sama. Skala rasio terdapat pada age, capital.gain, capital.loss, dan hours.per.week karena memiliki nol mutlak; misalnya, capital.gain = 0 berarti tidak ada keuntungan modal. Sementara itu, fnlwgt merupakan bobot survei, dan income adalah target klasifikasi dengan kategori <=50K dan >50K.

4. Pemrosesan Data

4.1 Import Library

Mengimpor library yang diperlukan untuk pengolahan data, preprocessing, pembagian dataset, pemodelan, dan evaluasi. Selain itu, kode mengatur tampilan data agar lebih mudah dibaca serta menentukan file dataset adult.csv dan kolom target income yang akan diprediksi.

Kode Python:

```python
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler, KBinsDiscretizer
from sklearn.naive_bayes import GaussianNB, CategoricalNB
from sklearn.metrics import (accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay)

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 200)


DATA_PATH = "adult.csv"
TARGET = "income"
```

✓ 6.2s

4.2 Memanggil Data

Output menunjukkan bahwa dataset memiliki 32.561 baris dan 15 kolom yang berisi informasi individu, seperti usia (age), pekerjaan (workclass), pendidikan (education), jenis kelamin (sex), dan jam kerja per minggu (hours.per.week). Lima baris pertama ditampilkan sebagai sampel data. Kolom income merupakan variabel target yang menunjukkan kategori pendapatan, yaitu <=50K atau >50K. Selain itu, terdapat nilai ? pada beberapa kolom yang mengindikasikan data yang perlu diperiksa lebih lanjut pada tahap preprocessing.

Kode Python:

```python
df_raw = pd.read_csv(DATA_PATH)
df = df_raw.copy()

print("Ukuran data (baris, kolom):", df.shape)
print(df.head())
```

✓ 0.2s

Output:

```
Ukuran data (baris, kolom): (32561, 15)
   age workclass  fnlwgt     education  education.num marital.status         occupation   relationship   race     sex  capital.gain  capital.loss  hours.per.week native.country income
0   90         ?   77053       HS-grad              9        Widowed                  ?  Not-in-family  White  Female             0          4356              40  United-States  <=50K
1   82   Private  132870       HS-grad              9        Widowed    Exec-managerial  Not-in-family  White  Female             0          4356              18  United-States  <=50K
2   66         ?  186061  Some-college             10        Widowed                  ?      Unmarried  Black  Female             0          4356              40  United-States  <=50K
3   54   Private  140359       7th-8th              4       Divorced  Machine-op-inspct      Unmarried  White  Female             0          3900              40  United-States  <=50K
4   41   Private  264663  Some-college             10      Separated     Prof-specialty      Own-child  White  Female             0          3900              40  United-States  <=50K
```

4.3 Info dan Tipe Kolom

df.info() menunjukkan bahwa dataset memiliki 32.561 baris dan 15 kolom, dengan seluruh kolom berisi 32.561 data non-null. Tipe data terdiri dari 6 kolom numerik (int64) dan 9 kolom kategorikal (object). Penggunaan memori sebesar 3,7 MB. Informasi ini digunakan untuk memahami struktur dan tipe data dataset sebelum dilakukan preprocessing. Perlu diperhatikan bahwa nilai ? yang muncul pada output sebelumnya tidak terdeteksi sebagai nilai kosong (null) oleh df.info(), sehingga perlu diperiksa secara terpisah.

Kode Python:



Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32561 entries, 0 to 32560
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   age             32561 non-null  int64
 1   workclass       32561 non-null  object
 2   fnlwgt          32561 non-null  int64
 3   education       32561 non-null  object
 4   education.num   32561 non-null  int64
 5   marital.status  32561 non-null  object
 6   occupation      32561 non-null  object
 7   relationship    32561 non-null  object
 8   race            32561 non-null  object
 9   sex             32561 non-null  object
 10  capital.gain    32561 non-null  int64
 11  capital.loss    32561 non-null  int64
 12  hours.per.week  32561 non-null  int64
 13  native.country  32561 non-null  object
 14  income          32561 non-null  object
dtypes: int64(6), object(9)
memory usage: 3.7+ MB
```

4.4 Statistik Deskriptif

df.describe() menampilkan statistik deskriptif dari kolom numerik pada dataset, meliputi jumlah data (count), rata-rata (mean), standar deviasi (std), nilai minimum (min), kuartil 25%, 50%, dan 75%, serta nilai maksimum (max). Hasil menunjukkan bahwa rata-rata usia individu adalah sekitar 38,58 tahun, sedangkan rata-rata jam kerja mencapai 40,44 jam per minggu. Statistik ini digunakan untuk memahami distribusi data dan mengidentifikasi kemungkinan nilai ekstrem (outlier) sebelum proses preprocessing dilakukan.

Kode Python:



Output:

	age	fnlwgt	education.num	capital.gain	capital.loss	hours.per.week
count	32561.000000	3.256100e+04	32561.000000	32561.000000	32561.000000	32561.000000
mean	38.581647	1.897784e+05	10.080679	1077.648844	87.303830	40.437456
std	13.640433	1.055500e+05	2.572720	7385.292085	402.960219	12.347429
min	17.000000	1.228500e+04	1.000000	0.000000	0.000000	1.000000
25%	28.000000	1.178270e+05	9.000000	0.000000	0.000000	40.000000
50%	37.000000	1.783560e+05	10.000000	0.000000	0.000000	40.000000
75%	48.000000	2.370510e+05	12.000000	0.000000	0.000000	45.000000
max	90.000000	1.484705e+06	16.000000	99999.000000	4356.000000	99.000000

4.5 Koreksi Tipe

Tipe data pada dataset telah dikoreksi sesuai kategorinya. Kolom numerik menggunakan tipe data int64, sedangkan kolom kategorikal, seperti workclass, education, dan income, menggunakan tipe data category. Selain itu, nama kolom telah dirapikan dengan mengganti tanda titik (.) menjadi garis bawah (_). Perbaikan ini bertujuan agar tipe data lebih sesuai dan memudahkan proses preprocessing serta klasifikasi menggunakan algoritma Naive Bayes.

Kode Python:

```python
#merapikan nama kolom
df.columns = df.columns.str.replace(".", "_", regex=False)

#memastikankolom angka berisi angka semua (kalo ada teks aneh otomatis jadi NaN)
kolom_angka = ["age", "fnlwgt", "education_num", "capital_gain", "capital_loss", "hours_per_week"]
df[kolom_angka] = df[kolom_angka].apply(pd.to_numeric, errors="coerce")

#mendefinisikan tipe data kategori
tipe_kategori = {
    'workclass': 'category',
    'education': 'category',
    'marital_status': 'category',
    'occupation': 'category',
    'relationship': 'category',
    'race': 'category',
    'sex': 'category',
    'native_country': 'category',
    'income': 'category' # Ini adalah TARGET
}
df = df.astype(tipe_kategori)

#mendefinisikan daftar kolom numerik dan kategorik (dipakai di langkah selanjutnya)
num_cols = kolom_angka
cat_cols = [c for c in tipe_kategori if c != TARGET]

print("Tipe Data Setelah Dikoreksi:")
print(df.dtypes)
```

✓ 0.1s

Output:

```
Tipe Data Setelah Dikoreksi:
age                int64
workclass       category
fnlwgt             int64
education       category
education_num      int64
marital_status  category
occupation      category
relationship    category
race            category
sex             category
capital_gain       int64
capital_loss       int64
hours_per_week     int64
native_country  category
income          category
dtype: object
```

4.6 Missing Value dan Penanganannya

Karakter ? pada dataset telah diubah menjadi nilai kosong (missing value) menggunakan np.nan. Hasil pemeriksaan menunjukkan bahwa kolom workclass memiliki 1.836 data kosong, occupation sebanyak 1.843 data, dan native_country sebanyak 583 data, sedangkan kolom lainnya tidak memiliki data kosong.

Kode Python:

```
#mengganti karakter "?" menjadi format NaN (Not a Number)
df = df.replace("?", np.nan)

print('\nMissing Value:')
print(df.isnull().sum())
```

✓ 0.1s

Output:

```
Missing Value:
age                 0
workclass        1836
fnlwgt              0
education           0
education_num       0
marital_status      0
occupation       1843
relationship        0
race                0
sex                 0
capital_gain        0
capital_loss        0
hours_per_week      0
native_country    583
income              0
dtype: int64
```

Penanganan:

Setelah dilakukan pemeriksaan missing value, fungsi dropna() digunakan untuk menghapus seluruh baris yang mengandung data kosong. Hasilnya, jumlah data berkurang dari 32.561 menjadi 30.162 baris. Pemeriksaan ulang menunjukkan bahwa seluruh kolom sudah tidak memiliki missing value sehingga dataset siap digunakan untuk tahap preprocessing selanjutnya.

Kode Python:

```
#menghapus semua baris yang mengandung NaN
df = df.dropna()

#mengecek ukuran data dan sisa missing value
print(f"Ukuran data sekarang: {df.shape[0]} baris")
print('\nMissing Value setelah dropna:')
print(df.isnull().sum())
```

✓ 0.1s

Output:

```
Ukuran data sekarang: 30162 baris

Missing Value setelah dropna:
age               0
workclass         0
fnlwgt            0
education         0
education_num     0
marital_status    0
occupation        0
relationship      0
race              0
sex               0
capital_gain      0
capital_loss      0
hours_per_week    0
native_country    0
income            0
dtype: int64
```

4.7 Duplikasi Data

Terdapat 23 baris data duplikat yang terdeteksi dan dihapus menggunakan drop_duplicates(). Setelah penghapusan dan pengaturan ulang indeks dengan reset_index(), dataset memiliki 30.139 baris dan 15 kolom. Proses ini bertujuan menghindari pengulangan data agar dataset lebih bersih dan siap digunakan pada tahap preprocessing selanjutnya.

Kode Python:

```python
#menghitung jumlah data yang duplikat
jumlah_duplikat = df.duplicated().sum()
print("Jumlah baris duplikat:", jumlah_duplikat)

#mnghapus baris yang duplikat dan mereset ulang nomor urut (index)
df = df.drop_duplicates().reset_index(drop=True)

#mengecek sisa baris dan kolom setelah dihapus
print("Ukuran data setelah hapus duplikat:", df.shape)
```

✓ 0.1s

Output:

```
Jumlah baris duplikat: 23
Ukuran data setelah hapus duplikat: (30139, 15)
```

4.8 Mengecek Ejaan atau Kapitalisasi

Seluruh kolom kategorikal dan target income telah diseragamkan menjadi huruf kecil serta dihilangkan spasi di awal dan akhir menggunakan strip() dan lower(). Hasilnya, ditampilkan jumlah nilai unik pada setiap kolom, seperti workclass sebanyak 7 nilai, education sebanyak 16 nilai, dan income sebanyak 2 kategori, yaitu <=50k dan >50k.

Kode Python:

```python
#menyamakan semua kolom kategorik dan target menjadi huruf kecil dan tanpa spasi di awal/akhir
for c in cat_cols + [TARGET]:
    df[c] = df[c].str.strip().str.lower()

for c in cat_cols + [TARGET]:
    print(f"{c} ({df[c].nunique()} nilai unik): {sorted(df[c].dropna().unique())}\n")
```

✓ 0.4s

Output:

```
workclass (7 nilai unik): ['federal-gov', 'local-gov', 'private', 'self-emp-inc', 'self-emp-not-inc', 'state-gov', 'without-pay']

education (16 nilai unik): ['10th', '11th', '12th', '1st-4th', '5th-6th', '7th-8th', '9th', 'assoc-acdm', 'assoc-voc', 'bachelors', 'doctorate', 'hs-grad', 'masters', 'preschool', 'prof-school', 'some-college']

marital_status (7 nilai unik): ['divorced', 'married-af-spouse', 'married-civ-spouse', 'married-spouse-absent', 'never-married', 'separated', 'widowed']

occupation (14 nilai unik): ['adm-clerical', 'armed-forces', 'craft-repair', 'exec-managerial', 'farming-fishing', 'handlers-cleaners', 'machine-op-inspct', 'other-service', 'priv-house-serv', 'prof-specialty', 'pro

relationship (6 nilai unik): ['husband', 'not-in-family', 'other-relative', 'own-child', 'unmarried', 'wife']

race (5 nilai unik): ['amer-indian-eskimo', 'asian-pac-islander', 'black', 'other', 'white']

sex (2 nilai unik): ['female', 'male']

native_country (41 nilai unik): ['cambodia', 'canada', 'china', 'columbia', 'cuba', 'dominican-republic', 'ecuador', 'el-salvador', 'england', 'france', 'germany', 'greece', 'guatemala', 'haiti', 'holand-netherlands

income (2 nilai unik): ['<=50k', '>50k']
```

Penanganan Kesalahan Penulisan Data dan Rekapitulasi Pendapatan:
Terdapat 4 kesalahan penulisan pada kolom native_country yang dikoreksi ejaannya. Jumlah kategori tetap 41 karena perbaikan hanya mengganti nama, tidak menggabungkan kategori. Selain itu, ditampilkan pula hasil perhitungan dari kolom income yang menunjukkan bahwa terdapat 22.633 data untuk kategori pendapatan di bawah atau sama dengan 50 ribu dan 7.506 data untuk kategori pendapatan di atas 50 ribu.

Kode Python:

```python
#penanganan kesalahan penulisan di kolom native_country
fix_country = {
    "holand-netherlands": "netherlands",
    "trinadad&tobago": "trinidad&tobago",
    "hong": "hong-kong",
    "columbia": "colombia",
}
df["native_country"] = df["native_country"].replace(fix_country)
print("Jumlah kategori native_country setelah perbaikan:", df["native_country"].nunique())
print(df[TARGET].value_counts())
```

✓ 0.0s

Output:

```
Jumlah kategori native_country setelah perbaikan: 41
income
<=50k    22633
>50k      7506
Name: count, dtype: int64
```

4.9 Outlier IQR pada Kolom Numerik
Mendeteksi data pencilan (outlier) pada beberapa kolom tertentu (age, hours_per_week, capital_gain, dan capital_loss) menggunakan metode IQR atau Interquartile Range. Cara kerjanya adalah dengan menghitung nilai kuartil pertama (Q1) dan kuartil ketiga (Q3) untuk menentukan batas bawah serta batas atas data normal, di mana nilai yang berada di luar rentang tersebut akan dihitung sebagai outlier. Hasil akhirnya disusun ke dalam bentuk tabel yang merangkum nama kolom, batas bawah, batas atas, jumlah data outlier, serta persentasenya terhadap keseluruhan

data. Berdasarkan tabel tersebut, kolom hours_per_week memiliki jumlah outlier terbanyak yaitu 7.947 data atau sekitar 26,37%, sedangkan kolom age memiliki outlier paling sedikit yaitu 168 data atau 0,56%.

Kode Python:

```python
#mengecek outlier pakai metode IQR
kolom_cek = ["age", "hours_per_week", "capital_gain", "capital_loss"]

#membuat list kosong buat nampung hasil tiap kolom
hasil = []

for kolom in kolom_cek:
    # hitung Q1, Q3, dan IQR
    Q1 = df[kolom].quantile(0.25)
    Q3 = df[kolom].quantile(0.75)
    IQR = Q3 - Q1

    # batas normal data
    batas_bawah = Q1 - 1.5 * IQR
    batas_atas = Q3 + 1.5 * IQR

    # hitung data yang di luar batas = outlier
    jumlah_outlier = len(df[(df[kolom] < batas_bawah) | (df[kolom] > batas_atas)])
    persen = round(jumlah_outlier / len(df) * 100, 2)

    # simpan ke list
    hasil.append([kolom, batas_bawah, batas_atas, jumlah_outlier, persen])

#mengubah list jadi tabel
tabel_outlier = pd.DataFrame(hasil, columns=["kolom", "batas_bawah", "batas_atas", "jumlah_outlier", "persen"])
print(tabel_outlier)
```

✓ 0.0s

Output:

	kolom	batas_bawah	batas_atas	jumlah_outlier	persen
0	age	-0.5	75.5	168	0.56
1	hours_per_week	32.5	52.5	7947	26.37
2	capital_gain	0.0	0.0	2538	8.42
3	capital_loss	0.0	0.0	1427	4.73

Penanganan:

Setelah outlier terdeteksi pada tahap sebelumnya, kode yang ini bertindak untuk memperbaikinya menggunakan metode clipping khusus pada kolom age dan hours_per_week. Alih-alih menghapus baris datanya, metode .clip() bekerja dengan cara mengganti nilai yang berada di luar batas normal agar masuk kembali tepat di garis batas bawah atau batas atasnya. Melalui fungsi describe() di akhir kode, terlihat bahwa nilai minimum dan maksimum kini sudah terkunci pas pada batas normal IQR yang telah ditentukan.

Kode Python:

```python
#penanganan outlier: nilai di luar batas diganti dengan nilai batasnya (clipping)
for kolom in ["age", "hours_per_week"]:
    Q1 = df[kolom].quantile(0.25)
    Q3 = df[kolom].quantile(0.75)
    IQR = Q3 - Q1
    batas_bawah = Q1 - 1.5 * IQR
    batas_atas = Q3 + 1.5 * IQR

    df[kolom] = df[kolom].clip(lower=batas_bawah, upper=batas_atas)

print(df[["age", "hours_per_week"]].describe())
```

✓ 0.0s

Output:

```
                age  hours_per_week
count  30139.000000    30139.000000
mean      38.408673       41.420336
std       13.024807        6.146161
min       17.000000       32.500000
25%       28.000000       40.000000
50%       37.000000       40.000000
75%       47.000000       45.000000
max       75.500000       52.500000
```

4.10 Ketidakseimbangan Kelas

Menampilkan distribusi data pada kolom target (income) dalam dua bentuk, yaitu jumlah absolut dan persentase. Hasilnya menunjukkan bahwa kategori <=50k berjumlah 22.633 data atau 75,1%, sedangkan kategori >50k berjumlah 7.506 data atau 24,9%.

Kode Python:

```
counts = df[TARGET].value_counts()
print(counts)
print("--------------------------------")
print((counts / counts.sum() * 100).round(2).astype(str) + " %")
```

✓ 0.0s

Output:

```
income
<=50k    22633
>50k      7506
Name: count, dtype: int64
--------------------------------
income
<=50k    75.1 %
>50k     24.9 %
Name: count, dtype: object
```

4.11 Indikasi Kebocoran Data

Kode tersebut digunakan untuk melakukan analisis fitur dan pembersihan tahap lanjut, dengan rincian sebagai berikut:

- Pemeriksaan Redundansi Pendidikan: Memeriksa kesamaan informasi antara kolom education dan education_num, yang menghasilkan nilai True karena keduanya merepresentasikan hal yang sama.
- Korelasi Fitur dengan Target: Menghitung tingkat korelasi antara kolom numerik dengan target y (>50k), di mana education_num memiliki korelasi tertinggi (0.34) sedangkan fnlwgt hampir tidak memiliki korelasi (-0.01).
- Pemeriksaan Duplikasi Fitur: Mendeteksi adanya 23 baris data yang memiliki nilai fitur identik (selain kolom target).
- Penghapusan Fitur Redundan: Menghapus kolom education dan fnlwgt dari dataset serta memperbarui daftar kolom numerik dan kategorikal agar tidak dipakai dalam pemodelan berikutnya.

Kode Python:

```python
#Memeriksa informasi yang sama atau berulang
check = pd.crosstab(df["education"], df["education_num"])
print("Education redundan:",
      (check.gt(0).sum(axis=1) == 1).all())

#Memeriksa korelasi fitur dengan target
tmp = df.copy()
tmp["y"] = (tmp[TARGET] == ">50k").astype(int)

print("\nKorelasi fitur dengan target:")
print(tmp[num_cols + ["y"]].corr()["y"].drop("y").round(2))

#memeriksa duplikasi fitur
print("\nJumlah baris dengan fitur identik:",
      df.drop(columns=[TARGET]).duplicated(keep=False).sum())

#menghapus informasi yang sama atau berulang dan fitur yang tidak digunakan
df = df.drop(columns=["education", "fnlwgt"], errors="ignore")
num_cols = [c for c in num_cols if c != "fnlwgt"]
cat_cols = [c for c in cat_cols if c != "education"]
```

✓ 0.4s

Output:

```
Education redundan: True

Korelasi fitur dengan target:
age               0.24
fnlwgt           -0.01
education_num     0.34
capital_gain      0.22
capital_loss      0.15
hours_per_week    0.27
Name: y, dtype: float64

Jumlah baris dengan fitur identik: 23
```

4.12 Data Hasil Pembersihan (5 Teratas dan 5 Terbawah)

Kode tersebut digunakan untuk menampilkan sampel isi dataset yang terdiri dari 5 baris data teratas dan 5 baris data terbawah setelah seluruh proses pembersihan data selesai.

- 5 Data Teratas (df.head(5)): Menampilkan baris data dari indeks 0 hingga 4 untuk memverifikasi struktur awal dari dataset yang sudah bersih.
- 5 Data Terbawah (df.tail(5)): Menampilkan 5 baris data terakhir dari indeks 30134 hingga 30138 guna memastikan kondisi penutup dari dataset.
- Kondisi Kolom Saat Ini: Kolom-kolom yang ditampilkan sudah bersih dan bebas dari fitur redundan yang sebelumnya dihapus (seperti education dan fnlwgt), sehingga data siap untuk tahap pemodelan.

Kode Python:

```python
print("=== 5 DATA TERATAS (setelah pembersihan) ===")
print(df.head(5))
print("\n=== 5 DATA TERBAWAH (setelah pembersihan) ===")
print(df.tail(5))
```

✓ 0.0s

Output:

```
=== 5 DATA TERATAS (setelah pembersihan) ===
    age workclass  education_num marital_status         occupation   relationship   race     sex  capital_gain  capital_loss  hours_per_week native_country income
0  75.5   private              9        widowed    exec-managerial  not-in-family  white  female             0          4356            32.5  united-states  <=50k
1  54.0   private              4       divorced  machine-op-inspct      unmarried  white  female             0          3900            40.0  united-states  <=50k
2  41.0   private             10      separated     prof-specialty      own-child  white  female             0          3900            40.0  united-states  <=50k
3  34.0   private              9       divorced      other-service      unmarried  white  female             0          3770            45.0  united-states  <=50k
4  38.0   private              6      separated       adm-clerical      unmarried  white    male             0          3770            40.0  united-states  <=50k

=== 5 DATA TERBAWAH (setelah pembersihan) ===
       age workclass  education_num      marital_status         occupation   relationship   race     sex  capital_gain  capital_loss  hours_per_week native_country income
30134  22.0   private             10       never-married    protective-serv  not-in-family  white    male             0             0            40.0  united-states  <=50k
30135  27.0   private             12  married-civ-spouse       tech-support           wife  white  female             0             0            38.0  united-states  <=50k
30136  40.0   private              9  married-civ-spouse  machine-op-inspct        husband  white    male             0             0            40.0  united-states   >50k
30137  58.0   private              9             widowed       adm-clerical      unmarried  white  female             0             0            40.0  united-states  <=50k
30138  22.0   private              9       never-married       adm-clerical      own-child  white    male             0             0            32.5  united-states  <=50k
```

4.13 Encoding

Melakukan proses encoding, yaitu mengubah data teks atau kategorikal di dalam dataset menjadi bentuk angka agar dapat diproses oleh algoritma pemodelan mesin. Pada bagian awal, variabel target (income) diubah menjadi angka biner, di mana kategori >50k bernilai 1 dan sisanya bernilai 0. Selanjutnya, variabel fitur X disiapkan dengan mengambil seluruh kolom selain target, lalu setiap kolom teks yang ada diubah menjadi kode numerik menggunakan fungsi .astype("category").cat.codes. Bagian akhir menampilkan hasil berupa 5 baris data teratas yang menunjukkan bahwa seluruh kolom teks seperti pekerjaan, status pernikahan, hingga jenis kelamin kini telah sepenuhnya berubah menjadi angka.

Kode Python:

```python
#proses encoding: ubah kolom kategorik (teks) jadi angka
# target dibuat angka dulu: <=50k = 0, >50k = 1
y = (df[TARGET] == ">50k").astype(int)

# fitur = semua kolom kecuali target
X = df.drop(columns=[TARGET]).copy()

# ubah tiap kolom teks jadi kode angka
for kolom in cat_cols:
    X[kolom] = X[kolom].astype("category").cat.codes

print("Contoh hasil encoding (5 data teratas):")
print(X.head())
print("\nTipe data setelah encoding:")
print(X.dtypes)
```

✓ 0.1s

Output:

```
Contoh hasil encoding (5 data teratas):
    age  workclass  education_num  marital_status  occupation  relationship  race  sex  capital_gain  capital_loss  hours_per_week  native_country
0  75.5          2              9               6           3             1     4    0             0          4356            32.5              38
1  54.0          2              4               0           6             4     4    0             0          3900            40.0              38
2  41.0          2             10               5           9             3     4    0             0          3900            40.0              38
3  34.0          2              9               0           7             4     4    0             0          3770            45.0              38
4  38.0          2              6               5           0             4     4    1             0          3770            40.0              38

Tipe data setelah encoding:
age               float64
workclass            int8
education_num       int64
marital_status       int8
occupation           int8
relationship         int8
race                 int8
sex                  int8
capital_gain        int64
capital_loss        int64
hours_per_week    float64
native_country       int8
dtype: object
```

4.14 Membagi Data Training dan Data Testing

Membagi dataset menjadi data latih (training) dan data uji (testing) menggunakan fungsi train_test_split dengan rasio 80% untuk data training dan 20% untuk data testing. Penggunaan parameter stratify=y memastikan bahwa proporsi kelas target pada data training dan testing tetap seimbang dan sama persis dengan dataset aslinya. Berdasarkan bagian output, terlihat bahwa data training memiliki 24.111 baris dan data testing memiliki 6.028 baris, dengan proporsi kelas 0 sebesar 75,1% dan kelas 1 sebesar 24,9% pada kedua bagian data tersebut.

Kode Python:

```python
# Bagi data: 80% training, 20% testing
# stratify=y supaya proporsi kelas di train dan test sama
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

print("Data training :", X_train.shape)
print("Data testing  :", X_test.shape)

print("\nProporsi kelas data training:")
print(y_train.value_counts(normalize=True).round(3))
print("\nProporsi kelas data testing:")
print(y_test.value_counts(normalize=True).round(3))
```

✓ 0.0s

Output:

```
Data training : (24111, 12)
Data testing  : (6028, 12)

Proporsi kelas data training:
income
0    0.751
1    0.249
Name: proportion, dtype: float64

Proporsi kelas data testing:
income
0    0.751
1    0.249
Name: proportion, dtype: float64
```

4.15 Normalisasi dan Standarisasi

Melakukan normalisasi dan standardisasi pada fitur numerik yaitu age, education_num, capital_gain, capital_loss, dan hours_per_week. Pada Min-Max Scaling, nilai setiap fitur diubah ke rentang 0–1, sehingga pada output terlihat nilai min = 0 dan max = 1; sedangkan StandardScaler mengubah data agar memiliki mean ≈ 0 dan standar deviasi = 1, terlihat dari output mean ≈ 0 dan std = 1. fit_transform() digunakan pada data training untuk mempelajari skala sekaligus mengubah datanya, sedangkan transform() digunakan pada data testing dengan skala yang sudah dipelajari dari training. Jadi, hasil output menunjukkan bahwa kedua metode berhasil diterapkan, dengan Min-Max menghasilkan rentang 0–1, sementara standardisasi menghasilkan data yang berpusat di 0 dengan standar deviasi 1.

Kode Python:

```python
# Normalisasi Min-Max (0-1) sebagai perbandingan
mm = MinMaxScaler()
X_train_mm = X_train.copy()
X_train_mm[num_cols] = mm.fit_transform(X_train[num_cols])

# Standarisasi (mean 0, std 1)
scaler = StandardScaler()
X_train_s, X_test_s = X_train.copy(), X_test.copy()
X_train_s[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test_s[num_cols] = scaler.transform(X_test[num_cols])

print("\nSetelah NORMALISASI Min-Max (min=0, max=1):")
print(X_train_mm[num_cols].describe().loc[["mean", "std", "min", "max"]].round(3))
print("Setelah STANDARISASI (mean~0, std~1):")
print(X_train_s[num_cols].describe().loc[["mean", "std", "min", "max"]].round(3))
```

✓ 0.1s

Output:

```
Setelah NORMALISASI Min-Max (min=0, max=1):
        age  education_num  capital_gain  capital_loss  hours_per_week
mean  0.367          0.608         0.011         0.020           0.446
std   0.223          0.170         0.076         0.092           0.307
min   0.000          0.000         0.000         0.000           0.000
max   1.000          1.000         1.000         1.000           1.000
Setelah STANDARISASI (mean~0, std~1):
        age  education_num  capital_gain  capital_loss  hours_per_week
mean -0.000         -0.000        -0.000        -0.000           0.000
std   1.000          1.000         1.000         1.000           1.000
min  -1.644         -3.578        -0.148        -0.218          -1.450
max   2.839          2.305        13.066        10.602           1.804
```

4.16 Data hasil praproses akhir: 5 teratas dan 5 terbawah

Membuat data akhir hasil praproses dari data training dengan menyalin X_train_s yang sudah distandardisasi, lalu menambahkan kolom income sebagai target yang diubah menjadi kode angka, yaitu 0 untuk <=50K dan 1 untuk >50K. Setelah itu, head(5) menampilkan 5 data teratas dan tail(5) menampilkan 5 data terbawah. Pada output terlihat bahwa kolom numerik seperti age, education_num, capital_gain, capital_loss, dan hours_per_week sudah dalam bentuk standardisasi, sedangkan kolom kategori seperti workclass, marital_status, occupation, dan lainnya sudah berupa kode angka. Jadi, hasil tersebut menunjukkan bahwa data training sudah melalui tahap praproses dan siap digunakan untuk proses klasifikasi.

Kode Python:

```python
final_train = X_train_s.copy()
final_train[TARGET] = y_train.map({0: "<=50k", 1: ">50k"})

print("=== 5 DATA TERATAS (hasil praproses akhir, data latih) ===")
print(final_train.head(5))
print("\n=== 5 DATA TERBAWAH (hasil praproses akhir, data latih) ===")
print(final_train.tail(5))
```

✓ 0.0s

Output:

```
=== 5 DATA TERATAS (hasil praproses akhir, data latih) ===
            age  workclass  education_num  marital_status  occupation  relationship  race  sex  capital_gain  capital_loss  hours_per_week  native_country income
10521 -1.260459          2      -0.048105               4          12             3     4    1     -0.147593     -0.217634       -0.229745              38  <=50k
10894  0.808348          4      -0.048105               2           2             0     4    1     -0.147593     -0.217634        1.804371              30  <=50k
9731  -0.877346          2       2.305328               4           0             3     4    0     -0.147593     -0.217634       -0.229745              38  <=50k
7348   1.038215          0      -0.440343               0           0             1     4    0     -0.147593     -0.217634       -0.229745              38  <=50k
16967  1.421327          3      -0.048105               2          11             0     4    1     -0.147593     -0.217634        1.397548              38   >50k

=== 5 DATA TERBAWAH (hasil praproses akhir, data latih) ===
            age  workclass  education_num  marital_status  occupation  relationship  race  sex  capital_gain  capital_loss  hours_per_week  native_country income
20777 -1.490326          2      -0.440343               4           7             3     4    1     -0.147593     -0.217634       -1.450215              38  <=50k
9967   0.271991          2      -0.440343               5           0             1     4    1     -0.147593     -0.217634       -0.229745              38  <=50k
2545   0.195368          2      -0.048105               2           2             0     0    1      0.816747     -0.217634        1.397548              38   >50k
26068  1.727817          2      -0.048105               4          12             3     4    1     -0.147593     -0.217634       -1.450215              38  <=50k
20435 -0.494234          1       1.128612               2           0             5     4    0     -0.147593     -0.217634       -0.229745              38  <=50k
```

4.17 Model Naive Bayes

A. GaussianNB

Membuat model Gaussian Naive Bayes dengan mengubah fitur kategori menjadi one-hot encoding, kemudian fitur numerik distandardisasi menggunakan StandardScaler. Setelah itu, GaussianNB() dilatih menggunakan data training dan digunakan untuk memprediksi data testing. Fungsi evaluate() menghitung akurasi, precision, recall, dan F1-score. Hasilnya menunjukkan akurasi 0,5629 atau 56,29%. Untuk kelas <=50K, precision-nya 0,97 tetapi recall hanya 0,43, sedangkan kelas >50K memiliki precision 0,36 dan recall 0,96. Artinya, model ini masih kurang baik dan kurang seimbang dalam melakukan klasifikasi, terutama karena asumsi Gaussian kurang cocok dengan bentuk data yang digunakan.

Kode Python:

```python
def evaluate(name, y_true, y_hat):
    print(f"===== {name} =====")
    print("Akurasi:", round(accuracy_score(y_true, y_hat), 4))
    print(classification_report(y_true, y_hat, target_names=["<=50k", ">50k"]))

# --- Model 1: GaussianNB (one-hot + standarisasi) ---
X_oh = pd.get_dummies(df.drop(columns=[TARGET]), columns=cat_cols, drop_first=True, dtype=int)
Xtr_oh, Xte_oh = X_oh.loc[X_train.index].copy(), X_oh.loc[X_test.index].copy()
sc2 = StandardScaler()
Xtr_oh[num_cols] = sc2.fit_transform(Xtr_oh[num_cols])
Xte_oh[num_cols] = sc2.transform(Xte_oh[num_cols])

gnb = GaussianNB().fit(Xtr_oh, y_train)
pred_gnb = gnb.predict(Xte_oh)
evaluate("GaussianNB (one-hot + standarisasi)", y_test, pred_gnb)
```

✓ 0.3s

Output:

```
===== GaussianNB (one-hot + standarisasi) =====
Akurasi: 0.5629
              precision    recall  f1-score   support

       <=50k       0.97      0.43      0.60      4527
        >50k       0.36      0.96      0.52      1501

    accuracy                           0.56      6028
   macro avg       0.67      0.70      0.56      6028
weighted avg       0.82      0.56      0.58      6028
```

B. CategoricalNB

Categorical Naive Bayes, yaitu fitur numerik terlebih dahulu diubah menjadi beberapa kategori menggunakan binning sebanyak 8 kelompok dengan KBinsDiscretizer, kemudian digabung dengan fitur kategori. Model CategoricalNB dilatih menggunakan data tersebut dan hasil prediksinya dievaluasi dengan fungsi yang sama. Hasilnya jauh lebih baik dengan akurasi 0,7946 atau 79,46%. Untuk kelas <=50K, precision 0,91, recall 0,80, dan F1-score 0,85, sedangkan kelas >50K memiliki precision 0,56, recall 0,77, dan F1-score 0,65. Jadi, CategoricalNB memberikan performa yang lebih baik daripada GaussianNB pada dataset ini.

Kode Python:

```
# --- Model 2: CategoricalNB (kategorik + numerik di-binning) ---
kb = KBinsDiscretizer(n_bins=8, encode="ordinal", strategy="quantile")
Xtr_c, Xte_c = X_train.copy(), X_test.copy()
Xtr_c[num_cols] = kb.fit_transform(X_train[num_cols]).astype(int)
Xte_c[num_cols] = kb.transform(X_test[num_cols]).astype(int)

# jumlah kategori minimum per fitur (agar kategori yang tidak muncul di data latih tidak error)
min_cats = [df[c].nunique() if c in cat_cols else 8 for c in Xtr_c.columns]

cnb = CategoricalNB(alpha=1.0, min_categories=min_cats).fit(Xtr_c, y_train)
pred_cnb = cnb.predict(Xte_c)
evaluate("CategoricalNB (kategorik + binning)", y_test, pred_cnb)
✓ 0.3s
```

Output:

```
===== CategoricalNB (kategorik + binning) =====
Akurasi: 0.7946
              precision    recall  f1-score   support

       <=50k       0.91      0.80      0.85      4527
        >50k       0.56      0.77      0.65      1501

    accuracy                           0.79      6028
   macro avg       0.74      0.79      0.75      6028
weighted avg       0.83      0.79      0.80      6028
```

Hasil: kedua model merupakan varian dari algoritma Naive Bayes, tetapi memberikan performa yang berbeda. GaussianNB menghasilkan akurasi sebesar 56,29%, sedangkan CategoricalNB menghasilkan akurasi sebesar 79,46%. CategoricalNB juga memiliki nilai F1-score yang lebih baik, yaitu 0,85 untuk kelas <=50K dan 0,65 untuk kelas >50K. Dengan demikian, CategoricalNB lebih cocok digunakan pada dataset Census Income ini karena mampu memberikan hasil klasifikasi yang lebih baik dibandingkan GaussianNB, terutama setelah fitur numerik diubah menjadi kategori melalui proses binning.

5. Kesimpulan

Laporan ini menerapkan klasifikasi pendapatan pada dataset Adult (Census Income) menggunakan algoritma Naive Bayes, dengan 32.561 baris data, 14 fitur prediktor, dan target income berkategori <=50K dan >50K. Pada tahap pemrosesan, nilai ? dihapus dan 23 baris duplikat dibuang sehingga tersisa 30.139 baris. Penulisan kategori diseragamkan, outlier pada age dan hours_per_week ditangani dengan clipping, serta kolom education dan fnlwgt dihapus karena redundan dan hampir tidak berkorelasi dengan target. Target ternyata tidak seimbang (75,1% untuk <=50K dan 24,9% untuk >50K), sehingga data dibagi 80:20 dengan stratify=y agar proporsi kelas tetap sama pada data latih dan data uji.

Pada tahap pemodelan, GaussianNB hanya mencapai akurasi 56,29% karena asumsi distribusi Gaussian kurang sesuai dengan data yang didominasi fitur kategorikal. Modelnya cenderung terlalu sering memprediksi kelas >50K, dengan recall 0,96 tetapi precision hanya 0,36. CategoricalNB, yang fitur numeriknya dikelompokkan ke dalam 8 bin, mencapai akurasi 79,46% dengan F1-score 0,85 untuk kelas <=50K dan 0,65 untuk kelas >50K. Dengan demikian, CategoricalNB lebih cocok untuk dataset ini, dan pemilihan varian Naive Bayes serta cara merepresentasikan fitur sangat memengaruhi hasil. Namun, performa pada kelas minoritas >50K masih terbatas dan akurasinya hanya sedikit di atas proporsi kelas mayoritas, sehingga F1-score dan recall perlu dilihat bersama akurasi. Hasil dapat ditingkatkan melalui penanganan ketidakseimbangan kelas, penyetelan jumlah bin dan parameter alpha, atau perbandingan dengan algoritma lain.

Referensi:

1. Becker, B., & Kohavi, R. (1996). Adult [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5XW20 (https://archive.ics.uci.edu/dataset/2/adult)

2. Kohavi, R. (1996). Scaling up the accuracy of Naive-Bayes classifiers: A decision-tree hybrid. Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96), 202-207. AAAI Press. https://www.osti.gov/biblio/421279